

SD Basic - String Matching and Replacing

Replacing, wrapping, comparing, pattern matching, quoting and phonetic matching.

This page continues SD Basic - String Functions.

Replacing

```
change(string, old, new {, count {, start})
swap(string, old, new {, count {, start})
substitute(array, old.list, new.list {, delimiter})
convert(from.characters, to.characters, string)
convert from.characters to to.characters in variable
```

`change()` and `swap()` are the same function — the compiler emits the same operation for both. Use whichever reads better; there is no behavioural difference to choose between.

The fourth and fifth arguments are frequently misread:

<i>count</i>	how many occurrences to change, not which one
<i>start</i>	which occurrence to start at

EXPRESSION ON <code>'BANANA'</code>	RESULT	
<code>change('banana', 'a', 'X')</code>	<code>bXnXnX</code>	all of them
<code>change('banana', 'a', 'X', 2)</code>	<code>bXnXna</code>	the first two
<code>change('banana', 'a', 'X', 1, 2)</code>	<code>banXna</code>	one, starting at the second
<code>change('banana', 'a', '')</code>	<code>bnn</code>	an empty replacement deletes

`CONVERT()` WORKS ON SINGLE CHARACTERS, AND DELETES

`convert()` maps each character of *from.characters* to the character in the same position of *to.characters*.

EXPRESSION

RESULT

`convert('abc', 'xyz', 'aabbcc')``xyyyzz` $a \rightarrow x, b \rightarrow y, c \rightarrow z$ `convert('abc', 'x', 'aabbcc')``xx`**b and c were deleted**

A character with no opposite number is removed, not left alone. If the two lists are not the same length, every character in the excess of *from.characters* is deleted from the string. This is the documented Pick behaviour and it is almost never what someone writing `convert(',', '', text)` expects to happen to the rest of their data — though it is exactly what they want when deleting is the intent.

`convert` also exists as a **statement**, which modifies a variable in place rather than returning a value:

```
convert ',', ' to ';' in line
```

SUBSTITUTE()

`substitute(array, old.list, new.list {, delimiter})` replaces whole **elements** of a dynamic array rather than characters within them, matching each element against *old.list*. Substituting `b` with `Z` in the values `a`, `b`, `c` gives `a`, `Z`, `c`.

Wrapping text into lines

```
fold(string, width {, delimiter})
```

Breaks *string* into lines of at most *width* characters, **breaking at spaces where it can** and hard-breaking where a single word is too long.

```
fold('the quick brown fox jumps over the lazy dog', 12, '|')
```

gives

```
the quick|brown fox|jumps over|the lazy dog
```

The two-argument and three-argument forms use different default separators, which is not an obvious thing to guess.

CALL	SEPARATOR
<code>fold(s, 12)</code>	field marks — four fields
<code>fold(s, 12, '')</code>	value marks — four values
<code>fold(s, 12, sep)</code>	whatever <i>sep</i> holds — its first character only

So passing an explicit empty delimiter is **not** the same as omitting it. If the mark level matters, give the delimiter explicitly.

The width may itself be a dynamic array, giving successive lines different lengths and reusing the last value once the list runs out:

```
fold(s, 5 : @vm : 20, '|')
```

gives `the|quick brown fox|jumps over the lazy|dog` — a five-character first line, then twenty. This is how you wrap a paragraph around an indent.

Comparing and testing

```
compare(string1, string2 {, justification})
alpha(string)
```

`compare()` returns `-1` if *string1* sorts first, `1` if *string2* does, and `0` if they are equal.

EXPRESSION	RESULT
<code>compare('abc', 'abd')</code>	<code>-1</code>
<code>compare('abd', 'abc')</code>	<code>1</code>
<code>compare('abc', 'abc')</code>	<code>0</code>
<code>compare('B', 'a')</code>	<code>-1</code> upper case sorts before lower — byte order
<code>compare('a2', 'a10')</code>	<code>1</code> plain text order: <code>2</code> after <code>1</code>

EXPRESSION	RESULT
<code>compare('a2', 'a10', 'r')</code>	<code>-1</code> right-justified: the numbers compare as numbers

The `r` justification is how you sort `item2` before `item10`. Without it, embedded numbers compare character by character and `10` sorts before `9`. `l` is the explicit left-justified form and is the default.

`alpha()` is true if every character is a letter.

EXPRESSION	RESULT
<code>alpha('abc')</code>	true
<code>alpha('ab1')</code>	false
<code>alpha('')</code>	false

`alpha('')` is false and `num('')` is true. The two tests disagree about the empty string, so a validation that uses both needs to say which answer it wants for an empty field rather than assuming they behave alike. `num()` is covered in SD Basic - Math Functions.

Pattern matching

string matches pattern
`matchfield(string, pattern, element)`

`matches` is an operator and returns true or false. `matchfield()` returns the part of the string that matched a given element of the pattern.

THE PATTERN LANGUAGE

ELEMENT	MATCHES
<code>na*</code>	exactly <i>n</i> alphabetic characters
<code>nn*</code>	exactly <i>n</i> digits
<code>nx*</code>	exactly <i>n</i> characters of any kind
<code>0a · 0n · 0x</code>	any number, including none

ELEMENT	MATCHES
$n - ma \cdot n - mn \cdot n - mx^*$	between n and m characters
"literal"	the quoted text, as it stands
$\sim element$	anything the element would not match

Elements are written one after another with no separator.

HOW THE CODES COMBINE

TEST	RESULT
'abc' matches '3a'	true
'ab12' matches '2a2n'	true
'abc' matches '0a'	true
' ' matches '0a'	true
'ab' matches '3x'	false
'12' matches '2-4n'	true
'1' matches '2-4n'	false
'12345' matches '2-4n'	false
'abc' matches '~3n'	true
'123' matches '~3n'	false
'ab-12' matches '2a"-2n'	true
'ab12' matches '0a0n'	true
'a1b2' matches '0a0n'	false

A pattern may be a dynamic array, and the values are alternatives. 'ab' matches ('2a' : @vm : '2n') is **true** — the string need only match one of them. This is how you accept several formats without writing the test three times:

```
valid = code matches ('3n' : @vm : '2a4n' : @vm : '"X"5n')
```

PICKING OUT THE PARTS

`matchfield(string, pattern, element)` returns the text that matched the *element*-th part of the pattern. On `'ab12'` against `'2a2n'`:

<code>matchfield('ab12', '2a2n', 1)</code>	<code>ab</code>
<code>matchfield('ab12', '2a2n', 2)</code>	<code>12</code>

That makes a pattern a parser as well as a test — validate with `matches`, then pull the pieces out with `matchfield()` rather than counting character positions by hand.

Quoting

```
quote(expr)
dquote(expr)
squote(expr)
```

<code>quote()</code> · <code>dquote()</code>	wrap in double quotes — <code>"a"</code>
<code>squote()</code>	wrap in single quotes — <code>'a'</code>

`quote()` and `dquote()` are the same function — the compiler emits the same operation for both, so there is nothing to choose between them.

For CSV output do not build quoting by hand: `csvdq()` and the CSV statements handle embedded quotes and separators to RFC 4180, and are covered in SD Basic - CSV Files.

Phonetic matching

```
soundex(string)
```

Returns a four-character code that is the same for words that sound alike, for finding a name someone has spelled by ear.

EXPRESSION	RESULT
<code>soundex('Smith')</code>	<code>S530</code>
<code>soundex('Smyth')</code>	<code>S530</code>
<code>soundex('Robert')</code>	<code>R163</code>
<code>soundex('Rupert')</code>	<code>R163</code>
<code>soundex('O Brien 123')</code>	<code>0165</code> — non-letters are ignored
<code>soundex('')</code>	<code>0000</code>

`soundex('')` returns `0000`, not an empty string. A blank name therefore gets a code, and every blank name gets the *same* code — so a search keyed on soundex will match all of them together unless you exclude empty input first.

Soundex is tuned for English and does poorly on names of other origins. It is a way to narrow a search, not to decide a match.

Applying a function to every element

Most functions here have a partner ending in `s` that applies the same operation to every element of a dynamic array:

<code>lens('one' : @vm : 'three')</code>	<code>3</code> and <code>5</code>
<code>counts('a-b' : @vm : 'c-d-e', '-')</code>	<code>1</code> and <code>2</code>
<code>fields('a-b' : @vm : 'c-d-e', '-', 1)</code>	<code>a</code> and <code>c</code>
<code>trims(' x ' : @vm : ' y ')</code>	<code>x</code> and <code>y</code>
<code>indexs('Ab' : @vm : 'cD', 'b', 1)</code>	<code>2</code> and <code>0</code>
<code>soundexs('Smith' : @vm : 'Smyth')</code>	<code>S530</code> and <code>S530</code>
<code>spaces(2 : @vm : 3)</code>	two spaces and three

The full family, and the rules for how they pair elements when given two arrays, is in SD Basic - Dynamic Arrays.

What is not here

`encrypt()` · `decrypt()` **removed** — replaced by `sdencrypt()` and `sddecrypt()`

Nothing else in the string-handling group has been removed. `dparse` and `dparse.csv` are statements rather than functions and are covered in SD Basic - Data Conversion and SD Basic - CSV Files; `iconv()`, `oconv()`, `fmt()`, `char()`, `seq()`, `ascii()` and `ebcdic()` are in SD Basic - Data Conversion; `extract()`, `insert()`, `delete()`, `replace()`, `locate`, `remove`, `raise()`, `lower()`, `vslice()` and `dcount()` used against marks are in SD Basic - Dynamic Arrays.

See also

SD Basic - Dynamic Arrays · SD Basic - Data Conversion · SD Basic - Math Functions · SD Basic - CSV Files.